JGI JAIN DEEMED-TO-BE UNIVERSITY | Powered By Futurense

Revolutionizing B.Tech

JGI JAIN DEEMED-TO-BE UNIVERSITY | Powered By Futurense

Course Name: Object Oriented Programming

Total Hours : 45

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Module 2: Classes and Constructors

Course Name: Object Oriented Programming [XXXXXX]

Total Hours : 10

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Table of Content

- Aim – Understanding the purpose and efficacy of object oriented programming
- Introduction
- Objective – unit wise split into 3 to 5
- Sections/Topics
- Self Assessments – 3 per section
- Activities – 1 per section
- Did You Know*
- Summary
- Terminal Questions
- Reference Links*
- Q&A*
- Thank You

*not mandatory

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Aim



To equip students in the fundamentals and understanding of object oriented paradigm and its efficacy in programming large projects

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense



Objective

a. Explain the fundamentals of software engineering to understand the need of object oriented programming

b. Understanding the java language fundamentals to implement object oriented programming

c. Understanding the components of Java language and its various keywords to create an object oriented paradigm

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Course Outcome (Cos)

On completion of the course the student will be able to:

CO1: Understand Java syntax, object-oriented principles, and control structures.
CO2: Implement object-oriented concepts using classes, objects, and constructors.
CO3: Apply inheritance, polymorphism, and dynamic method dispatch in Java.
CO4: Design modular Java applications using interfaces, abstract classes, and packages.
CO5: Handle exceptions, file I/O, and apply collections in Java applications.
CO6: Analyze and implement advanced Java programming features such as multi-threading, interfaces, wrapper classes, and package structuring for efficient, modular, and concurrent application development.

JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Module 2

Classes and Constructors – Classes - Declaration, object state, behaviour, Instance vs static members, Constructors - Default and parameterized constructors, Constructor overloading, Copy constructors, Method Overloading - Compile-time polymorphism, Rules for overloading, Static Members - static variables and methods, Static blocks, main method, this and final - Resolving ambiguity using this, Using final with variables, methods, classes

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Classes and Objects

- **Class (The Blueprint):**
  - A template or blueprint from which objects are created.
  - It defines a new data type.
  - Syntax: class ClassName { ... }
- **Object (The Instance):**
  - A physical entity that represents memory allocation of a class.
  - State: Represents data (value of variables).
  - Behavior: Represents functionality (methods).
- Key Exam Point: A class is a logical entity; an object is a physical entity (occupies memory in Heap).

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Classes and Objects

```
class Student
{
    int rollNumber;
    String name;
    String department;

    public void displayDetails()
    {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Department: " + department);
    }

    public void study()
    {
        System.out.println(name + " is studying Java
programming");
    }
}
```

```
class ClassDemo1
{
    public static void main(String[] args)
    {
        Student s1 = new Student();
        s1.rollNumber = 101;
        s1.name = "Rahul";
        s1.department = "CSE";

        s1.displayDetails();
        s1.study();
    }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Instance vs. Static Members

Feature	Instance Variables	Static Variables
Scope	Belongs to a specific object.	Belongs to the class (common to all).
Memory	Allocated whenever a new object is created.	Allocated only once when the class is loaded.
Access	Accessed via object reference (obj.var).	Accessed via class name (Class.var).
Default	Has default values (0, null, etc.).	Has default values.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Instance vs. Static Members

```
class Student
{
    int rollNumber;
    String name;
    String department;

    public void displayDetails()
    {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Department: " + department);
    }

    public void study()
    {
        System.out.println(name + " is studying Java
programming");
    }
}
```

```
class ClassDemo2 {
    public static void main(String[] args) {

        Employee e1 = new Employee();
        e1.empId = 201;
        e1.empName = "Anita";
        Employee e2 = new Employee();
        e2.empId = 202;
        e2.empName = "Ravi";

        e1.display();
        System.out.println();
        e2.display();
        System.out.println();
        Employee.showCompanyPolicy();
    }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructors

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructors

- A special block of code initialized when an object is created.
- A constructor is a special method in a class used to initialize objects.
- **Rules:**
  - Must have the same name as the class.
  - Must not have a return type (not even void).
- **Types:**
  - Default Constructor: No arguments. Added by compiler if no constructor is written.
  - Parameterized Constructor: Accepts arguments to initialize objects with specific values.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Types of Constructors

1. **Default Constructor:**
   - Created by the compiler if no constructor is defined.
2. **Parameterized Constructor:**
   - Allows passing arguments to initialize object attributes.
3. **Copy Constructor** (Java doesn't have this explicitly but can be mimicked using parameterized constructors).

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

1.1 Simple Default Constructor

```
class Bike
{
    // Default Constructor
    Bike()
    {
        System.out.println("Bike is created");
    }

    public static void main(String args[])
    {
        // Calling a default constructor
        Bike b = new Bike();
    }
}
```

Output:

Bike is created

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

1.2 Simple Default Constructor

```
class Constructor1
{
	int a;
	String b;
	double c;
	Constructor1()
	{

	}
	public static void main(String[] args)
	{
		Constructor1 obj = new Constructor1();
		System.out.println("Value of a is: " + obj.a);
		System.out.println("Value of a is: " + obj.b);
		System.out.println("Value of a is: " + obj.c);
	}
}
```

Output:

Value of a is: 0
Value of a is: null
Value of a is: 0.0

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

1.3 Default Constructor

```
class Student
{
    int rollNumber;
    String name;

    Student()
    {
        rollNumber = 100;
        name = "Default Student";
    }

    public void display()
    {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
    }
}
```

```
class DefaultConstructorDemo
{
    public static void main(String[] args)
    {
        Student s1 = new Student();
        s1.display();
    }
}
```

Output:

```
Roll Number: 100
Name: Default Student
```

JGI JAIN DEEMED-TO-BE UNIVERSITY | Powered By Futurense

2.1 Parameterized Constructor

```
class Student
{
    int id;
    String name;

    // Parameterized Constructor
    Student(int i, String n)
    {
        id = i;
        name = n;
    }
    void display()
    {
        System.out.println(id + " " + name);
    }
```

```
public static void main(String args[])
    {
        // Creating objects and passing values
        Student s1 = new Student(111, "Karan");
        Student s2 = new Student(222, "Aryan");

        s1.display();
        s2.display();
    }
}
```

Output:

```
111 Karan
222 Aryan
```

JGI JAIN DEEMED-TO-BE UNIVERSITY | Powered By Futurense

2.2 Parameterized Constructor

```
class Employee
{
    int empId;
    String empName;

    Employee(int id, String name)
    {
        empId = id;
        empName = name;
    }
    public void display()
    {
        System.out.println("Employee ID: " + empId);
        System.out.println("Employee Name: " +
empName);
    }
}
```

```
class ParameterizedConstructorDemo
{
    public static void main(String[] args)
    {
        Employee e1 = new Employee(301, "Anita
Roy");
        e1.display();
    }
}
```

Output:

```
Employee ID: 301
Employee Name: Anita
Roy
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Copy Constructors

- A constructor that initializes an object using another object of the same class.
- Purpose: To create a "clone" or exact copy of an existing object.
- Java Note: Unlike C++, Java does not define a default copy constructor; you must define it explicitly.
  - Example: Student(Student s) { this.name = s.name; }

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

3. Copy Constructor

```
class Employee {

    int empId;
    String empName;

    Employee(int id, String name) {
        empId = id;
        empName = name;
    }

    Employee(Employee obj) {
        empId = obj.empId;
        empName = obj.empName;
    }

    public void display() {
        System.out.println("Employee ID: " + empId);}
        System.out.println("Employee Name: " + empName);
    }
}
```

```
class CopyConstructorDemo {
    public static void main(String[] args) {

        Employee e1 = new Employee(401, "Rahul
Verma");

        Employee e2 = new Employee(e1);

        System.out.println("Original Object Details");
        e1.display();

        System.out.println();

        System.out.println("Copied Object Details");
        e2.display();
    }
```

JGI JAIN DEEMED-TO-BE UNIVERSITY | Powered By Futurense

3. Copy Constructor

Output:

Original Object Details
Employee ID: 401
Employee Name: Rahul Verma

Copied Object Details
Employee ID: 401
Employee Name: Rahul Verma

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Overloading

- **Definition:**
  - Constructor Overloading means having more than one constructor in a class with different parameter lists.
- **Purpose:**
  - To provide multiple ways to initialize an object with different sets of data.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Overloading – Part 1

```
class Student {

    int rollNumber;
    String name;
    String department;

    Student() {
        rollNumber = 0;
        name = "Unknown";
        department = "Not Assigned";
    }

    Student(int r, String n) {
        rollNumber = r;
        name = n;
        department = "General";
    }

    Student(int r, String n, String d) {
        rollNumber = r;
        name = n;
        department = d;
    }

    public void display() {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("Department: " + department);
        System.out.println();
    }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Overloading – Part 2

```
class ConstructorOverloadingDemo {
    public static void main(String[] args) {

        Student s1 = new Student();
        Student s2 = new Student(101, "Anita");
        Student s3 = new Student(102, "Rahul", "CSE");

        s1.display();
        s2.display();
        s3.display();
    }
}
```

Output:

Roll Number: 0
Name: Unknown
Department: Not Assigned

Roll Number: 101
Name: Anita
Department: General

Roll Number: 102
Name: Rahul
Department: CSE

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Chaining (Out of Syllabus)

- **Definition:**
  - Constructor Chaining is the process where one constructor calls another constructor of the same class or the superclass.
- **Purpose:**
  - To reuse constructor logic, reducing redundancy.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Chaining

Advantages of Constructor Chaining:

- **Reduces Code Duplication:** Allows reusing existing constructors to avoid redundant code.
- **Improves Readability:** Chaining constructors lead to more readable and organized code.
- **Easy Maintenance:** Easier to manage changes by modifying logic in one constructor instead of multiple places.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Chaining

```
class ConstructorChain {
    ConstructorChain() {
        System.out.println("No-arg Constructor");
    }

    ConstructorChain(String name) {
        this();
        System.out.println("Single-arg Constructor: " + name);
    }

    ConstructorChain(String name, int age) {
        this(name);
        System.out.println("Two-arg Constructor: Name = " + name + ", Age = " + age);
    }

    public static void main(String[] args) {
        ConstructorChain obj = new ConstructorChain("Alice", 25);
    }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Constructor Chaining

Output:

No-arg Constructor
Single-arg Constructor: Alice
Two-arg Constructor: Name = Alice, Age = 25

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Method Overloading

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Method Overloading (Compile-time Polymorphism)

- **Definition:**
  - Method Overloading is a feature in Java where a class has multiple methods with the same name but different parameters.
  - It is also known as Compile-Time Polymorphism or Static Binding.
- **Why is it called Compile-Time Polymorphism?**
  - The Java compiler determines which method to call at compile time based on the method signature (name + parameters).
  - If the compiler cannot find a matching method, it throws an error immediately.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Method Overloading (Compile-time Polymorphism)

- **Rule 1: Change the Number of Arguments**
    - Method A: add(int a, int b)
    - Method B: add(int a, int b, int c)
- **Rule 2: Change the Data Type of Arguments**
    - Method A: display(int a)
    - Method B: display(String a)
- **Rule 3: Change the Order of Arguments**
    - Method A: print(int a, String b)
    - Method B: print(String b, int a)

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Method Overloading – Program 1

```
class Demo
{
    void show(int a)
    {
        System.out.println("Integer value: " + a);
    }

    void show(double a)
    {
        System.out.println("Double value: " + a);
    }

    public static void main(String[] args) {
        Demo obj = new Demo();
        obj.show(10);
        obj.show(10.5);
    }
}
```

Output:

Integer value: 10
Double value: 10.5

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Method Overloading – Program 2

```
class Calculator
{
    void add(int a, int b)
    {
        System.out.println("Int Sum: " + (a + b));
    }

    void add(double a, double b)
    {
        System.out.println("Double Sum: " + (a + b));
    }

    void add(int a, int b, int c)
    {
        System.out.println("Triple Sum: " + (a + b + c));
    }
```

```
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        // 1. Calling Method 1 (int, int)
        calc.add(10, 20);

        // 2. Calling Method 2 (double, double)
        calc.add(5.5, 2.2);

        // 3. Calling Method 3 (int, int, int)
        calc.add(10, 20, 30);
    }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Static Variables (Class Variables)

- **Definition:**
  - A variable declared with the static keyword.
- **Key Characteristics:**
  - Memory Allocation: Memory is allocated only once when the class is loaded (in the Method Area), not every time an object is created.
  - Shared Property: All objects of the class share the same copy of the static variable.
  - Access: Can be accessed directly using the class name (e.g., Student.collegeName).

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Static Methods

- **Definition:**
  - A method declared with the static keyword.
- **Key Characteristics:**
  - No Object Needed: Can be called without creating an instance of the class (e.g., Math.sqrt()).
  - Access Restrictions:
    - Static methods can only access static data members and other static methods.
    - They cannot access instance variables or instance methods directly.
    - They cannot use this or super keywords.

JGI JAIN DEEMED-TO-BE UNIVERSITY | Powered By Futurense

Static Variable and Method

```
class Student
{
    int rollNumber;
    String name;
    static String collegeName = "ABC Engineering College";

    void display()
    {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
        System.out.println("College Name: " + collegeName);
        System.out.println();
    }
    static void showCollegeInfo()
    {
        System.out.println("All students belong to the same
college");
    }
}
```

```
class StaticVarMethod1
{
   public static void main(String[] args)
   {
      Student s1 = new Student();
      s1.rollNumber = 101;
      s1.name = "Rahul";

      Student s2 = new Student();
      s2.rollNumber = 102;
      s2.name = "Anita";

      s1.display();
      s2.display();
      System.out.println(Student.collegeName);
      Student.showCollegeInfo();
   }
}
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Static Variable and Method

Output:

Roll Number: 101
Name: Rahul
College Name: ABC Engineering College

Roll Number: 102
Name: Anita
College Name: ABC Engineering College

ABC Engineering College
All students belong to the same college

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Static Blocks & The Main Method

- **Static Block (static { ... }):**
  - Used to initialize static variables.
  - Executed automatically when the class is loaded into memory.
  - Runs before the main method.
- **The Main Method (public static void main):**
  - Why Public? So the JVM (Java Virtual Machine) can access it from outside.
  - Why Static? So the JVM can call it without creating an object of the class.
  - Why Void? It doesn't return any value to the JVM.
  - String args[]: Accepts command-line arguments.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Static Blocks

```
class StaticDemo2 {

    static int number;

    static {
        number = 50;
        System.out.println("Static Block Executed");
    }

    public static void main(String[] args) {

        System.out.println("Main Method Executed");
        System.out.println("Value of number: " + number);
    }
}
```

Output:

Static Block Executed
Main Method Executed
Value of number: 50

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

This Keyword

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Introduction to this Keyword

- **Definition:**
  - The this keyword in Java is a reference variable that points to the current object (the object that is calling the method or constructor).
- **Purpose:**
  - It differentiates between instance variables and parameters with the same name.
  - It allows calling another constructor in the same class.
  - It can return the current class instance from a method.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Introduction to this Keyword

Simple Explanation

- The this keyword refers to the current object.
- It is used to assign values to instance variables when local variables have the same name.

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Common Uses of this Keyword

- To refer to the current class instance variable.
- To invoke the current class constructor.
- To pass the current object as a parameter to a method.
- To pass the current object as an argument to the constructor.
- To return the current object from the method.

- **Syntax**:

```
this.instanceVariable;
```

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Use of this to Invoke a Constructor

Syntax:

- this(); // Calls another constructor in the same class

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

This Keyword

```
class Employee
{
    int id;
    String name;

    Employee(int id, String name)
    {
        this.id = id;
        this.name = name;
    }

    void show()
    {
        System.out.println("ID: " + this.id);
        System.out.println("Name: " + this.name);
    }
}
```

```
class ThisKeyword
{
    public static void main(String[] args)
    {
        Employee e = new Employee(101, "Anita");
        e.show();
    }
}
```

Output:

ID: 101
Name: Anita

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

This Keyword with Constructor

```
class Student
{
    int rollNumber;
    String name;
    Student()    {
        this(101, "Rahul");
    }
    Student(int r, String n)    {
        rollNumber = r;
        name = n;
    }
    void display()    {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Name: " + name);
    }
}
```

```
class ThisKeyword2
{
    public static void main(String[] args)
    {
        Student s1 = new Student();
        s1.display();
    }
}
```

Output:

ID: 101
Name: Anita

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

final Keyword

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Introduction to final Keyword

- **Definition:**
  - final is a non-access modifier used to restrict the user.
  - It implies "constant" or "unchangeable.“

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

final Keyword

1. Final Variable:

- Value becomes constant.
- Once assigned, it cannot be re-assigned.
- MUST be initialized when declared or inside a constructor.

2. Final Method:

- Cannot be overridden by a subclass.
- Prevents inheritance from modifying the method's logic.

3. Final Class:

- Cannot be inherited (cannot have child classes).

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

final Keyword

```
final class Student
{
    final int rollNumber = 101;

    final void display()
    {
        System.out.println("Roll Number: " + rollNumber);
        System.out.println("Student information cannot be
modified");
    }
}
```

```
class FinalDemo
{
    public static void main(String[] args)
    {
        Student s1 = new Student();
        s1.display();
    }
}
```

Output:

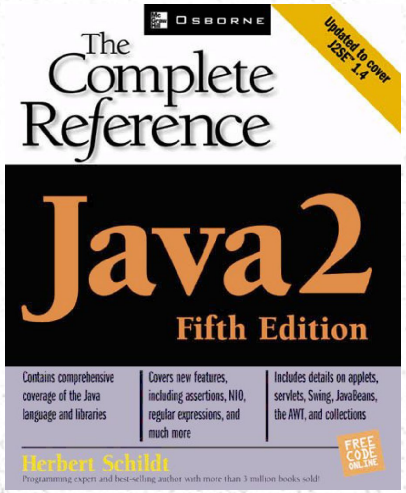
Roll Number: 101
Student information cannot be modified

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Reference Links

1. Java 2 Complete Reference 5th Edition - https://isa.atu.edu.iq/wp-content/uploads/2020/02/The-Complete-Reference-java-5th.pdf

2. Java 2 Complete Reference 7th Edition https://drive.google.com/file/d/1Y4jzNIUJ_3GhsiMpAm5lET09rhbOwizr/view?usp=sharing



JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense

Thank you

JGI JAIN DEEMED-TO-BE UNIVERSITY
Powered By Futurense